Index

Sr. No	Aim	Date	Page No	Signature
1	A) A simple client class that generates the private and public keys by using the built- in python RSA Algorithm and tests it. B) A Transaction class to send and receive the money.			
2	A) Create multiple transactions and display them. B) Create a block chain, a genesis block and execute it.			
3	Write a python program to create a mining function and test it.			
4	Implement and demonstrate the use of following in solidity a) Variables b) Operators c) Loops d) Decision Making			
5	Implementation the concept of structure (Struct) using solidity.			
6	Write a solidity program to implement emulator			
7	Implement view and pure function in solidity.			
8	Implement and demonstrate the use of function overload and mathematical function in solidity.			
9	Implement and Demonstrate the used of following in solidity: a) Inheritance b) Constructor			
10	Implementation and demonstrate the use following in solidity: a. Withdrawal Pattern b. Restricted access			

Practical No: 1

Aim: A) A simple client class that generates the private and public keys by using the built- in python RSA Algorithm and tests it.
B) A Transaction class to send and receive the money.

Input:

A) A simple client class that generates the private and public keys by using the built- in python RSA Algorithm and tests it.

```
>>> ! pip install pycryptodome
```

Code:

```
from Crypto import Random
from Crypto.Hash import SHA
from Crypto.PublicKey import RSA
from Crypto.Signature import PKCS1_v1_5
import binascii

class Client:
    def __init__(self):
        random_gen = Random.new().read
        self._private_key = RSA.generate(1024, random_gen)
        self._public_key = self._private_key.publickey()
        self._signer = PKCS1_v1_5.new(self._private_key)

    @property
    def identity(self):
        return binascii.hexlify(
            self._public_key.exportKey(format='DER')
        ).decode('ascii')

Alice = Client()
print(Alice.identity)
```

Output:

```
Shell
>>> %Run blockchain_prac_1a.py
 30819f300d06092a864886f70d010101050003818d0030818902818100b4d06382ce271e2aa30cd5dcb84bd2
 80340dc00a9817e6d80fd5b546ba0592835d2e2245b812fb2df70acbdc20c09c44bb871b85482f2176c711b2
 d61bbec69bbb2e30bfb544dc6c40e2501b0274d8a84bc3b364e23a59328d499a6d20acd4821cba8d5e98f14e
 1be1670e7311fa2c4d5d4435ce697a26ceb53afa8b2b7f8b1b0203010001
>>>
```

B) A Transaction class to send and receive the money

Code:

```
class Transaction:
    def __init__(self, account_holder, balance=0):
        self.account_holder = account_holder
        self.balance = balance

    def send_money(self, amount, recipient):
        if amount <= 0:
            return "Amount must be greater than zero"
        if amount > self.balance:
            return "Insufficient balance"

        self.balance -= amount
        recipient.receive_money(amount)
        return f"Sent {amount} to {recipient.account_holder}. Remaining balance:
{self.balance}."

    def receive_money(self, amount):
        if amount <= 0:
            return "Amount must be greater than zero."
        self.balance += amount
        return f"Received {amount}. New balance: {self.balance}."

    def __str__(self):
        return f"Account Holder: {self.account_holder}, Balance: {self.balance}"
```

```
# Creating accounts
account1 = Transaction("Alice", 1000)
account2 = Transaction("Bob", 300)

# Testing transaction
print(account1.send_money(300, account2))
print(account1)
print(account2)
```

Output:

```
>>> %Run Blockchain_pra_1.py
 Sent 300 to Bob. Remaining balance: 700.
 Account Holder: Alice, Balance: 700
 Account Holder: Bob, Balance: 600
```

Practical No: 2A

Aim: A) Create multiple transactions and display them

Input:

```
class Transaction:
        def __init__ (self, transaction_id, date, amount, description):
      self.transaction_id = transaction_id
      self.date = date
      self.amount = amount
      self.description = description

def __str__(self):
      return(f"Transaction ID: {self.transaction_id},"
            f"Date: {self.date}",
            f"Amount: ${self.amount:.2f},"
            f"Description: {self.description}")

def create_transaction():
      transaction_id = input("Enter transcation ID:")
        date = input("Enter transaction date (YYYY - MM- DD):")
        amount = float(input("Enter transaction amount:"))
      description = input("Enter transaction description:")
        return Transaction(transaction_id, date, amount, description)

def display_transactions(transactions):
        if not transactions:
          print("No transactions to display.")
        else:
          print("\nTransaction History:")
          for i, transaction in enumerate(transactions, start=1):
              print(f"{i}. {transaction}")

def main():
   transactions =[]
        while True:
      print("\nMenu:")
      print("1. Create a new transaction")
      print("2. Display all transactions")
```

```
    print("3. Exit")

      choice = input("Enter your choice:")
      if choice == '1':
        transaction=create_transaction()
        transactions.append(transaction)
        print("Transaction added successufully.")
      elif choice == '2':
        display_transactions(transactions)
      elif choice == '3':
        print("Exiting the program. Goodbuys!")
      break
      else:
        print("Invalid choice. Please try again")

if __name__ =="__main__":
      main()
```

Output:

```
>>> %Run 2a_blockchain.py

 Menu:
 1. Create a new transaction
 2. Display all transactions
 3. Exit
 Enter your choice:1
 Enter transcation ID:4731
 Enter transaction date (YYYY - MM- DD):2025 - 12 -05
 Enter transaction amount:70000
 Enter transaction description:order
 Transaction added successufully.
>>>
```

```
Shell
>>> %Run 2a_blockchain.py

 Menu:
 1. Create a new transaction
 2. Display all transactions
 3. Exit
 Enter your choice:2
 No transactions to display.
>>>
```

```
Shell
>>> %Run 2a_blockchain.py

 Menu:
 1. Create a new transaction
 2. Display all transactions
 3. Exit
 Enter your choice:3
 Exiting the program. Goodbuys!
>>>
```

Practical no: 2b

Aim: Create a block chain, a genesis block and execute it.

Input:

```
import hashlib
import time

class Block:
        def __init__(self, index, timestamp, data, previous_hash):
      self.index = index
      self.timestamp = timestamp
      self.data = data
      self.previous_hash = previous_hash
      self.hash = self.calculate_hash()

        def calculate_hash(self):

      block_string = (str(self.index) +
                str(self.timestamp) +
                str(self.data) +
                   str(self.previous_hash))
        return hashlib.sha256(block_string.encode()).hexdigest()


class Blockchain:
        def __init__(self):
      self.chain = [self.create_genesis_block()]

        def create_genesis_block(self):
        return Block(0, time.time(), "Genesis Block", "0")

        def get_latest_block(self):
        return self.chain[-1]

        def add_block(self, data):
      prev_block = self.get_latest_block()
      new_block = Block(
         index=prev_block.index + 1,
```

```
            timestamp=time.time(),
            data=data,
            previous_hash=prev_block.hash
          )
        self.chain.append(new_block)

if __name__ == "__main__":
          my_chain = Blockchain()
    my_chain.add_block("First transaction data")
    my_chain.add_block("Second transaction data")
    my_chain.add_block("Thrid transaction data")

    print("Blockchain Created: \n")
          for block in my_chain.chain:
        print("-------------------------------------")
        print(f"Block Index  :  {block.index}")
        print(f"Timestamp   :  {block.timestamp}")
        print(f"Data            :  {block.data}")
        print(f"Previous Hash:  {block.previous_hash}")
        print(f"Current Hash :  {block.hash}")
        print("_----------------------------\n")
```

Output:

```
Shell
>>> %Run 4_blockchain.py
 Blockchain Created:

 -------------------------------------
 Block Index  :  0
 Timestamp    :  1764922764.3718545
 Data         :  Genesis Block
 Previous Hash:  0
 Current Hash :  3fec401f8c6178a0673eae74416406a48a58cb2929a9de39e2a99bdc66189a3e
 _-----------------------------

 -------------------------------------
 Block Index  :  1
 Timestamp    :  1764922764.3718545
 Data         :  First transaction data
 Previous Hash:  3fec401f8c6178a0673eae74416406a48a58cb2929a9de39e2a99bdc66189a3e
 Current Hash :  c9b145b6219f89c0e5137aadc6388db84bde6f0d55cacc5c0d1515e94bf83d61
 _-----------------------------
```

```
-------------------------------------
Block Index  :  2
Timestamp    :  1764922764.3718545
Data         :  Second transaction data
Previous Hash:  c9b145b6219f89c0e5137aadc6388db84bde6f0d55cacc5c0d1515e94bf83d61
Current Hash :  4f7ab0f8d105f49fe9993f902e17d9920c537a64f262afc978f10588ab5bd325
_-----------------------------
_

-------------------------------------
Block Index  :  3
Timestamp    :  1764922764.3718545
Data         :  Thrid transaction data
Previous Hash:  4f7ab0f8d105f49fe9993f902e17d9920c537a64f262afc978f10588ab5bd325
Current Hash :  d3a0d7f688abd666240e0f5570376e875c99e94d729e94fff69d7df886e66b28
_-----------------------------
_

>>>
```

Practical No: 3

Aim: Write a python program to create a mining function and test it.

Code:

```
import hashlib
import time

def compute_hash(data, nonce):
    block_string = f"{data}{nonce}".encode()
    return hashlib.sha256(block_string).hexdigest()

def mine_block(data, difficulty):
    target = "0" * difficulty
    nonce = 0
    start_time = time.time()

    print(f"Mining started... Difficulty: {difficulty}")

    while True:
        hash_value = compute_hash(data, nonce)
        if hash_value.startswith(target):
            end_time = time.time()
            print(f"Block mined!")
            print(f"Nonce found: {nonce}")
            print(f"Hash  : {hash_value}")
            print(f"Timetaken: {end_time - start_time:.4f} second")
            return nonce, hash_value
        nonce +=1

if __name__ == "__main__":
    data = "Hello, this is my block data!"
    difficulty = 4

    nonce, block_hash = mine_block(data, difficulty)
```

```
print("\n Testing verification...")
test_hash = compute_hash(data, nonce)
print("Valid hash?", test_hash.startswith("0" * difficulty))
```

Output:

```
Shell

>>> %Run 3_blockchain.py
 Mining started... Difficulty: 4
 Block mined!
 Nonce found: 41682
 Hash  : 0000a5c2e038619276a75e313fe2ffe6dd9e41dcc5b40426ffd14ac0d554ac01
 Timetaken: 0.0160 second

  Testing verification...
 Valid hash? True
>>>
```

Practical No: 4

Aim: Implement and demonstrate the use of following in solidity

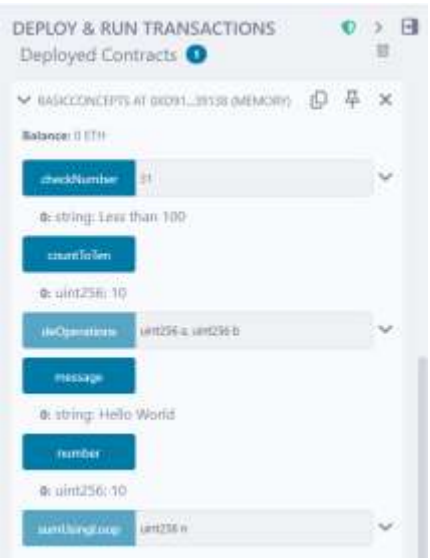
a) Variables
b) Operators
c) Loops
d) Decision Making

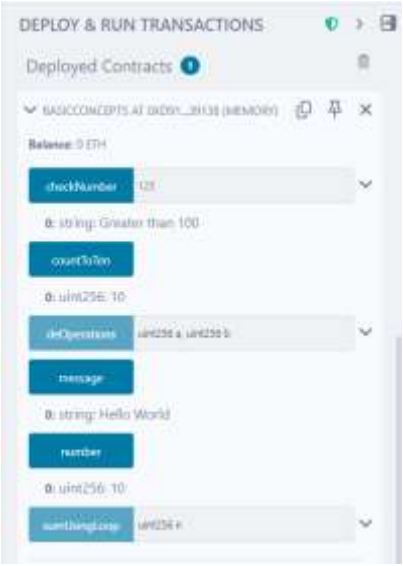
Code:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
contract   BasicConcepts {

    uint public number = 10;
        string public message = "Hello World";

        function deOperations(uint a, uint b) public pure returns (
        uint sum,
        uint product,
        bool isAGreater,
        bool bothPositive
        ) {
        sum = a + b;
      product = a * b;
      isAGreater = a > b;
      bothPositive = (a > 0 && b > 0);
        }

        function sumUsingLoop(uint n) public pure returns (uint) {
        uint total = 0;

        for (uint i = 1; i <=n; i++) {
         total += i;
           }
```

```
        return total;
        }

        function countToTen() public pure returns (uint) {
        uint x = 0;

        while (x < 10) {
          x++;
        }
        return x;
        }

        function checkNumber(uint value) public pure returns (string memory) {
        if (value > 100) {
          return "Greater than 100";
        } else if (value == 100) {
          return "Equal to 100";
        } else {
          return "Less than 100";
        }
        }
}
```

Output:





Practical No: 5

Aim: Implementation the concept of structure (Struct) using solidity

Code:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;
contract StructExample {

    struct Student {
        string name;
        uint age;
        uint rollNo;
    }

    Student public student;
    Student[] public students;

    function setStudent(
        string memory _name,
        uint _age,
        uint _rollNo
    ) public {
        student = Student ( _name, _age, _rollNo);
    }

    function addStudent (
        string memory _name,
        uint _age,
        uint _rollNo
    ) public {
        students.push(Student( _name, _age, _rollNo));
    }

    function getStudent(uint index)
    public
```

```
    view
    returns (Student memory)
    {
        return students[index];
    }

    function updateStudent(
        uint index,
        string memory _name,
        uint _Age,
        uint _rollNo
    ) public {
        students[index].name = _name;
        students[index].age = _Age;
        students[index].rollNo = _rollNo;
    }
}
```

Output:

Step 1: Using setStudent

Step 2: Using addStudent

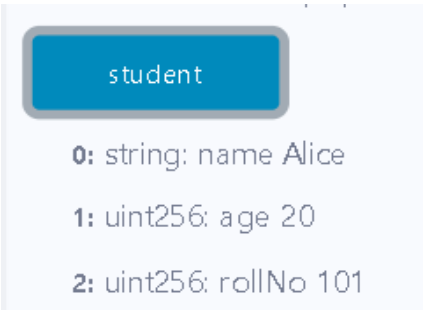
Step 3 : add another student using addStudent

Step 4: Using getStudent

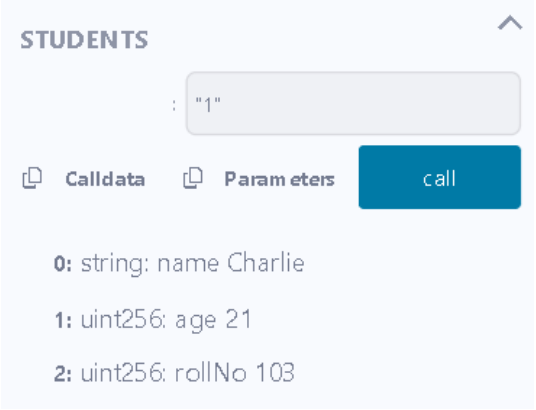
Step 5: Using updateStudent

Step 6: Now check again:
Call getStudent(0)

Step 7: click on Student



Step 8: students



Practical No: 6

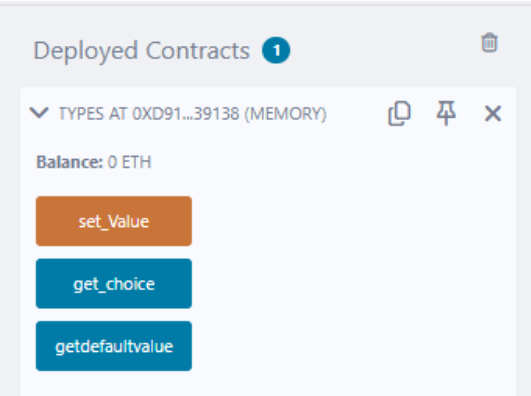
Aim: Write a solidity program to implement emulator

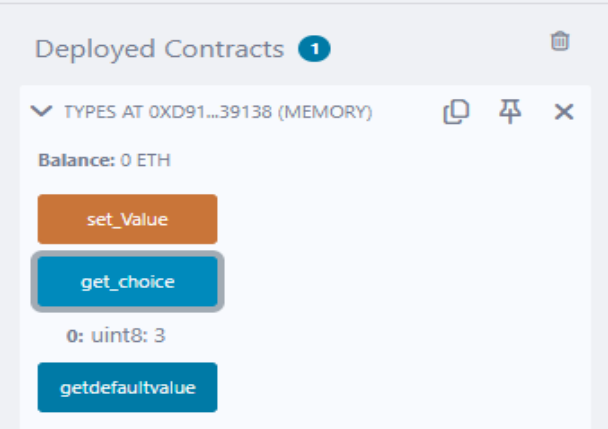
Code:

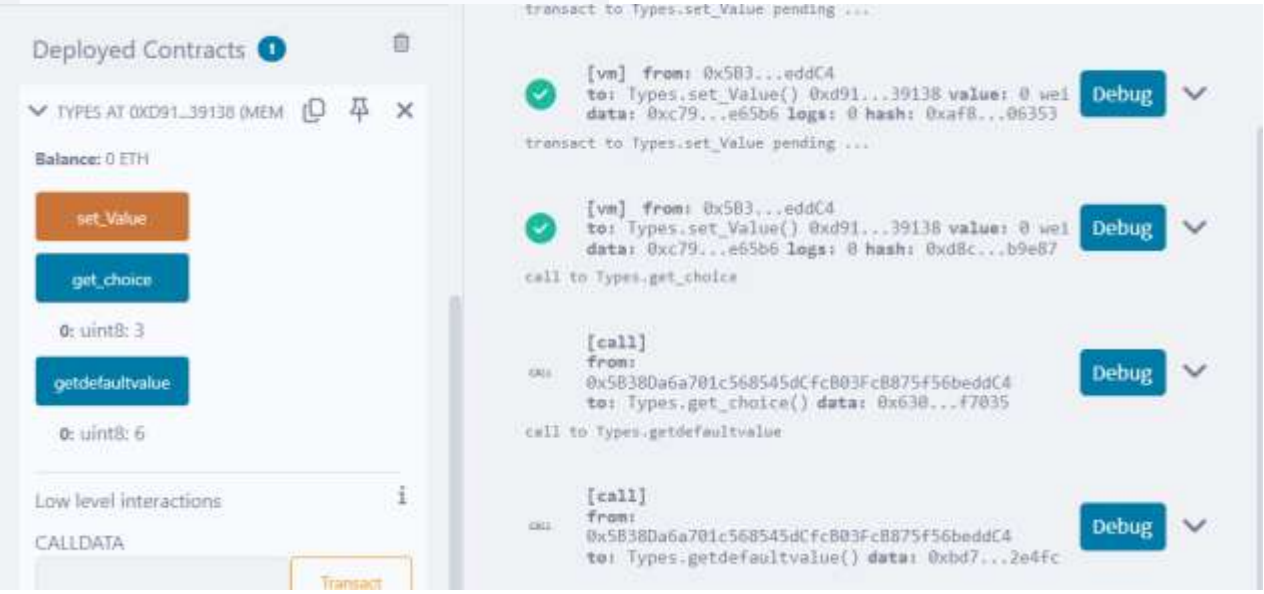
```
// SPDX-License-Identifier: MIT
pragma solidity ^0.5.0;
contract Types {

    enum week_days
    {
        Monday,
        Tuesday,
        Wednesday,
        Thursday,
        Friday,
        Saturaday,
        Sunday
    }

    week_days week;
    week_days choice;

    week_days constant default_value
     = week_days.Sunday;

     function set_Value() public {
        choice = week_days.Thursday;
     }

     function get_choice(
     ) public view returns (week_days) {
        return choice;
     }

     function getdefaultvalue(
```

```
    ) public pure returns(week_days){
      return default_value;
    }

}
```

Output:







Practical No: 7

Aim: Implement view and pure function in solidity

Code:

```
//SPDX-License-Identifier: GPL-3.0
// SPDX-License-Identifier: MIT
pragma solidity ^0.5.0;

contract Test {
    uint num1 = 2;
    uint num2 = 4;

    function getResult(
    ) public view returns(
        uint product, uint sum
    ){
        product = num1 * num2;
        sum = num1 + num2;
    }

}
```

Output:

Practical No: 8

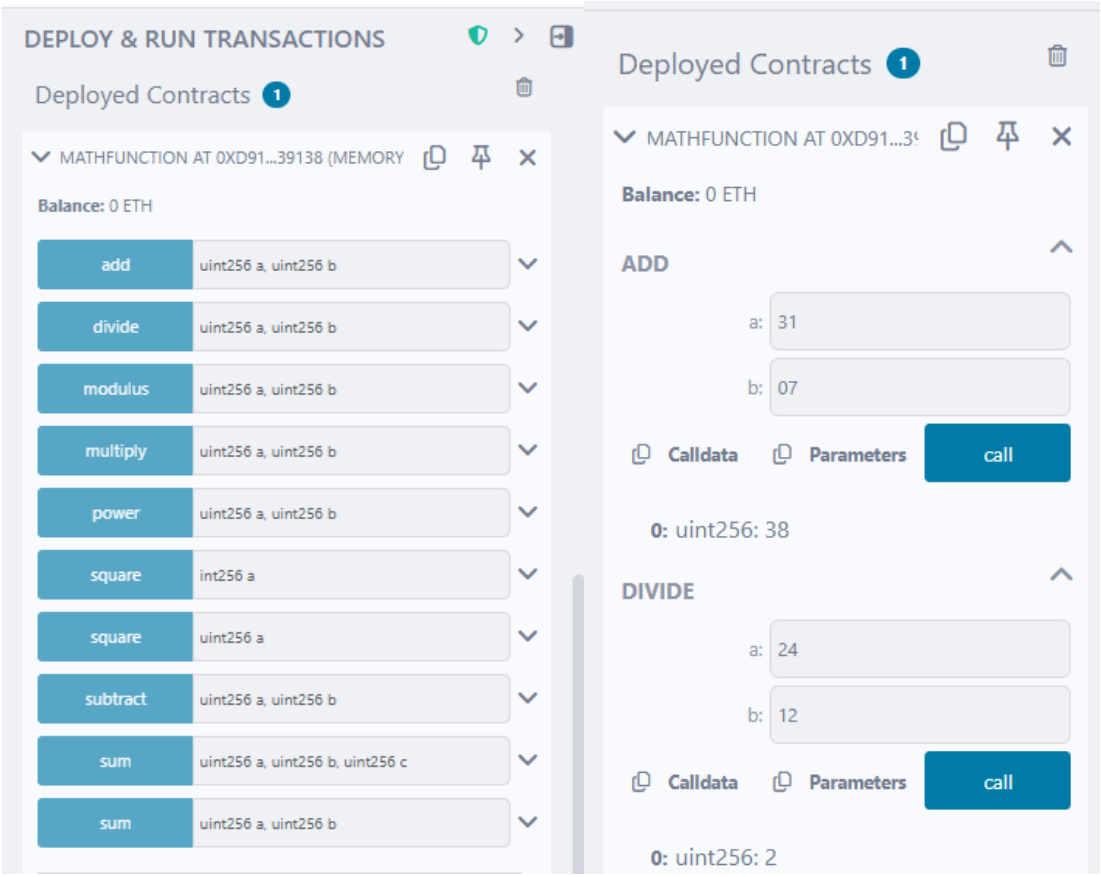
Aim: Implement and demonstrate the use of function overload and mathematical function in solidity.

Code:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract MathFunction {
    function add (uint a, uint b) public pure returns (uint) {
        return a + b;
    }

    function subtract (uint a, uint b) public pure returns (uint) {
        return a - b;
    }

    function multiply (uint a, uint b) public pure returns (uint) {
        return a * b;
    }

    function divide  (uint a, uint b) public pure returns (uint) {
        return a / b;
        // This will revert if b is 0
    }

    function modulus (uint a, uint b) public pure returns (uint) {
        return a % b;
        // This will revert if b is 0
    }

    function power (uint a, uint b) public pure returns (uint) {
        return a ** b;
    }
```

```
    function sum(uint a, uint b) public pure returns (uint) {
        return a + b;
    }

    function sum ( uint a, uint b, uint c) public pure returns (uint) {
        return a + b + c;
    }

     function square (uint a) public pure returns (uint) {
        return a * a;
    }

    function square (int a) public pure returns (int) {
        return a * a;
    }
}
```

Output:



MODULUS

a: 1

b: 7

Calldata Parameters call

0: uint256: 1

MULTIPLY

a: 7

b: 7

Calldata Parameters call

0: uint256: 49

POWER

a: 2

b: 2

Calldata Parameters call

0: uint256: 4

SQUARE

a: 12

Calldata Parameters call

0: int256: 144

SQUARE

a: 3

Calldata Parameters call

0: uint256: 9

SUBTRACT

a: 82

b: 24

Calldata Parameters call

0: uint256: 58

SUM

a: 31

b: 30

c: 05

Calldata Parameters call

0: uint256: 66

SUM

a: 6

b: uint256

Calldata Parameters call

0: uint256: 12

Practical No:9

Aim: Implement and Demonstrate the used of following in solidity:

a) Inheritance
b) Constructor

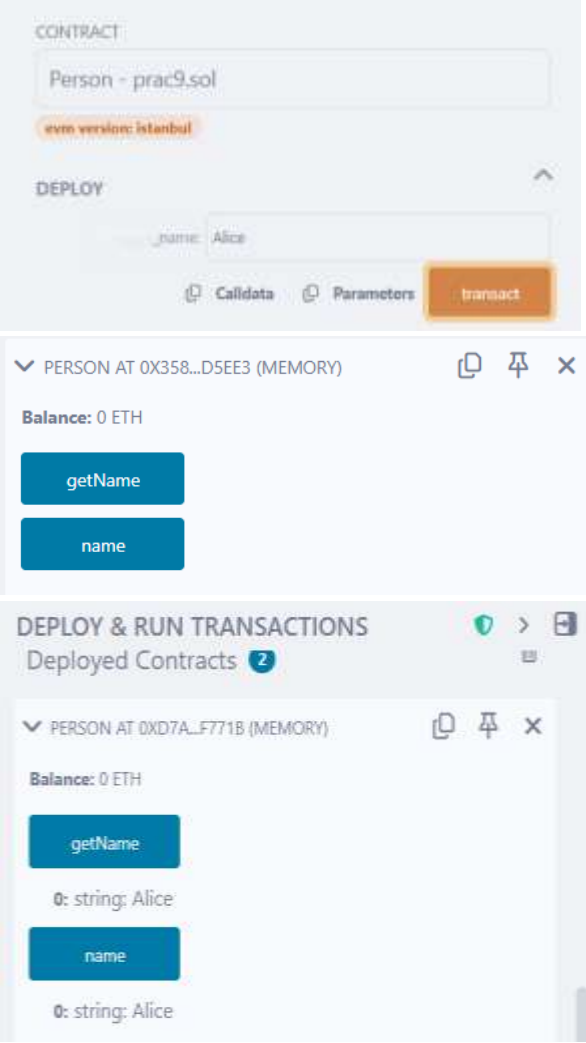
Input:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract Person {
    string public name;

    //Contructor of Person Contract
    constructor(string memory _name) {
        name = _name;
    }

    function getName() public view returns (string memory) {
        return name;
    }

}

contract Student is Person {
    uint public rollNo;

    constructor(string memory _name, uint _rollNo)
    Person(_name)
    {
        rollNo = _rollNo;
    }

    function getDetails() public view returns ( string memory, uint) {
        return (name, rollNo);
    }
```

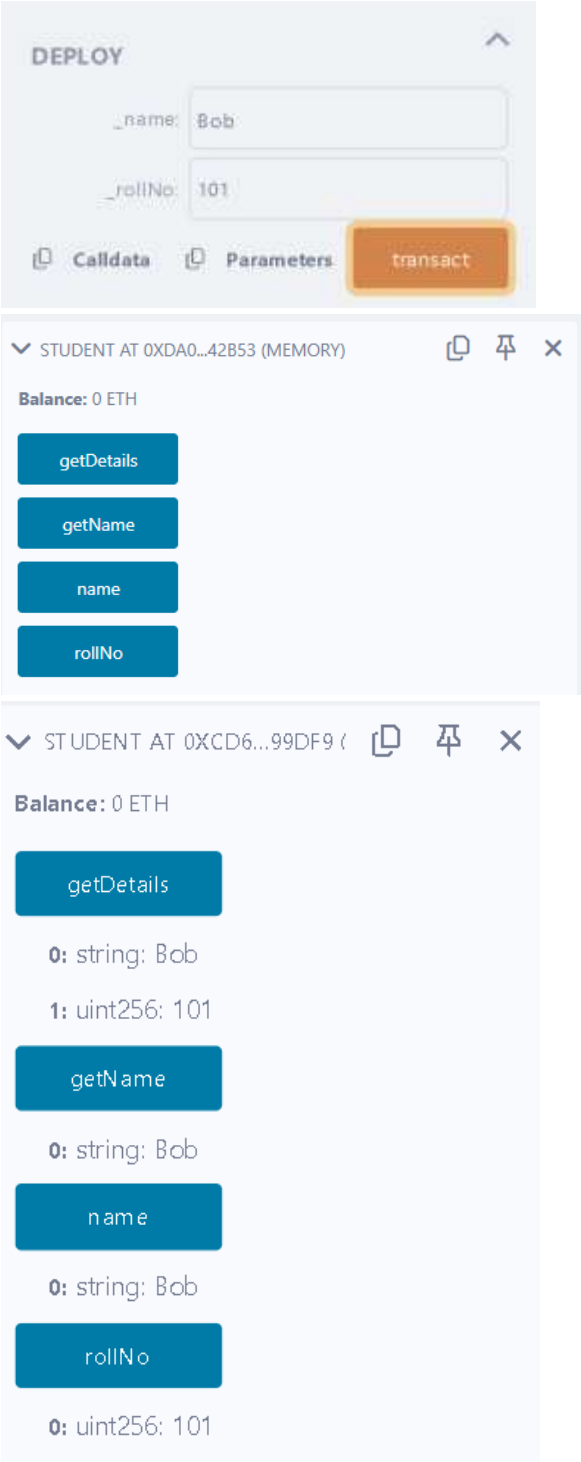
}

Output:

Person



Student



Practical No: 10

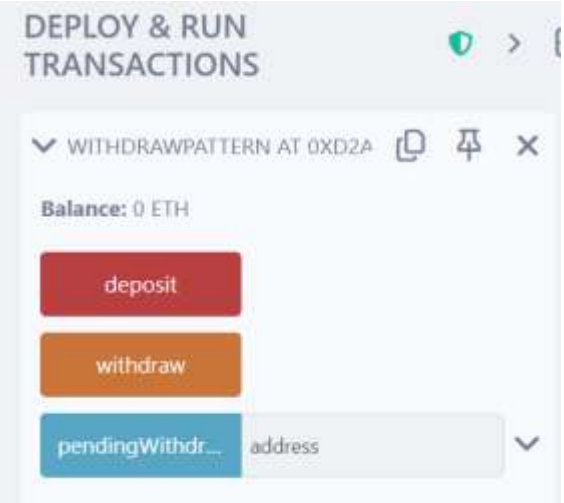
Aim: Implementation and demonstrate the use following in solidity:

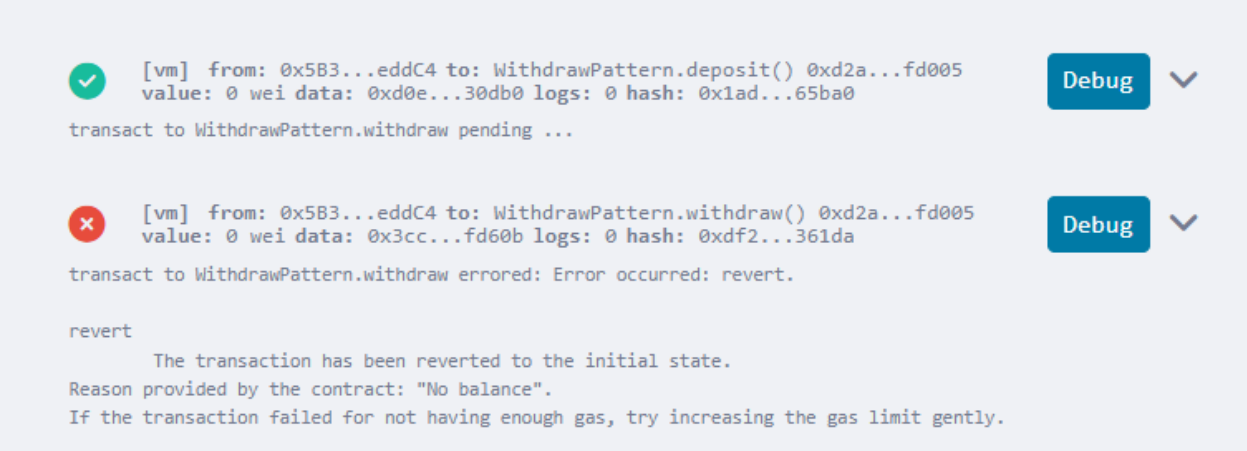
a) Withdrawal Pattern
b) Restricted access

Code:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract WithdrawPattern {
    mapping ( address => uint ) public pendingWithdrawals;

    function deposit() public payable {
        pendingWithdrawals[msg.sender] += msg.value;
    }

    function withdraw() public {
        uint amount = pendingWithdrawals[msg.sender];
        require(amount > 0, "No balance");

        pendingWithdrawals[msg.sender] = 0;

        payable(msg.sender).transfer(amount);
    }
}
```

Output:


DEPLOY & RUN TRANSACTIONS
WITHDRAWPATTERN AT 0XD2A
Balance: 0 ETH
deposit
withdraw
pendingWithdr...
address


[vm] from: 0x5B3...eddC4 to: WithdrawPattern.deposit() 0xd2a...fd005
value: 0 wei data: 0xd0e...30db0 logs: 0 hash: 0x1ad...65ba0
Debug
transact to WithdrawPattern.withdraw pending ...
[vm] from: 0x5B3...eddC4 to: WithdrawPattern.withdraw() 0xd2a...fd005
value: 0 wei data: 0x3cc...fd60b logs: 0 hash: 0xdf2...361da
Debug
transact to WithdrawPattern.withdraw errored: Error occurred: revert.
revert
The transaction has been reverted to the initial state.
Reason provided by the contract: "No balance".
If the transaction failed for not having enough gas, try increasing the gas limit gently.